JavaScript Variables

What is a Variable?

A **variable** is a named container used to store data in memory. It allows you to store, update, and reuse data in your program.

Example

```
let name = "Rokon";
const age = 23;

console.log(name);
console.log(age);
```

Output

```
Rokon
23
```

Types of JavaScript Variables

JavaScript has **3 types of variables**:

1. `var`
2. `let`
3. `const`

1. var

Explanation

- Old way of declaring variables.
- Value can be changed.

- Variable can be declared again.
- Function scoped.

Example

```
var city = "Dhaka";

console.log(city);

city = "Rajshahi";

console.log(city);

var city = "Khulna";

console.log(city);
```

Output

```
Dhaka
Rajshahi
Khulna
```

2. let

Explanation

- Modern way of declaring variables.
- Value can be changed.
- Cannot be declared again in the same scope.
- Block scoped.

Example

```
let score = 50;

console.log(score);

score = 80;

console.log(score);
```

Output

```
50
80
```

Wrong Example

```
let score = 50;
let score = 100; // Error
```

3. const

Explanation

- Modern way of declaring variables.
- Value cannot be changed after assignment.
- Cannot be declared again.
- Block scoped.

Example

```
const country = "Bangladesh";

console.log(country);
```

Output

```
Bangladesh
```

Wrong Example

```
const country = "Bangladesh";

country = "India"; // Error
```

Comparison

Feature	var	let	const
Reassign Value	Yes	Yes	No
Redeclare	Yes	No	No
Scope	Function	Block	Block
Recommended	No	Yes	Yes

Best Practice

- Use `const` by default.
- Use `let` if the value changes.
- Avoid `var` in modern JavaScript.

Summary

- `var` → Old method, can change and redeclare.
- `let` → Modern method, can change but cannot redeclare.
- `const` → Modern method, cannot change after assignment.

Variable Scope in JavaScript

Definition

Variable Scope is the area of a program where a variable can be accessed or used. It determines the visibility and lifetime of a variable.

Types of Variable Scope

1. Global Scope

A variable declared outside any function or block is called a **global variable**. It can be accessed from anywhere in the program.

Example

```
let name = "John";

function greet() {
  console.log(name);
}

greet();          // John
console.log(name); // John
```

2. Function Scope

A variable declared with `var` inside a function is only accessible within that function.

Example

```
function test() {
  var age = 20;
  console.log(age);
}

test();          // 20
console.log(age); // Error
```

3. Block Scope

A variable declared with `let` or `const` inside a block (`{}`) can only be accessed within that block.

Example

```
if (true) {
  let x = 10;
  const y = 20;

  console.log(x); // 10
  console.log(y); // 20
}

console.log(x); // Error
console.log(y); // Error
```

`var` vs `let` vs `const`

Keyword	Scope	Can be Reassigned	Can be Redeclared
`var`	Function Scope	Yes	Yes
`let`	Block Scope	Yes	No
`const`	Block Scope	No	No

Importance of Variable Scope

- Prevents variable name conflicts.
- Improves code readability.
- Makes code easier to maintain.
- Protects variables from being accessed outside their intended area.
- Helps reduce bugs and unexpected behavior.

Summary

Scope	Description
Global Scope	Accessible from anywhere in the program.
Function Scope	Accessible only inside the function where it is declared.
Block Scope	Accessible only inside the block `{}` where it is declared.

Key Points

- `var` → Function Scoped
- `let` → Block Scoped
- `const` → Block Scoped
- Global variables are accessible everywhere.
- Function variables cannot be accessed outside the function.
- Block-scoped variables cannot be accessed outside the block.

Interview Question

What is Variable Scope in JavaScript?

Answer:

Variable scope is the region of a program where a variable is accessible. JavaScript provides three main types of scope:

1. Global Scope
2. Function Scope
3. Block Scope

Variables declared with `let` and `const` are block-scoped, while variables declared with `var` are function-scoped.

JavaScript Data Types

JavaScript has **8 data types**. They are divided into **Primitive** and **Non-Primitive (Reference)** types.

Primitive Data Types

String

Represents text data. Written inside single (`' '`), double (`" "`), or backticks (`` ` ` ``).

```
let name = "Rokon";
let city = 'Dhaka';
```

Number

Represents both integer and floating-point numbers.

```
let age = 23;
let price = 99.99;
```

Boolean

Represents a logical value: `true` or `false` .

```
let isStudent = true;
let isLoggedIn = false;
```

Null

Represents an intentional empty or unknown value.

```
let data = null;
```

Undefined

A variable that has been declared but not assigned a value.

```
let score;
console.log(score); // undefined
```

BigInt

Used for very large integers beyond the safe `Number` limit.

```
let bigNumber = 1234567890123456789012345678890n;
```

Symbol

Represents a unique and immutable value.

```
let id = Symbol("id");
```

Non-Primitive Data Type

Object

Stores collections of data using key-value pairs.

```
const user = {
  name: "Rokon",
  age: 23,
  isStudent: true,
};
```

Summary Table

Data Type	Description	Example
String	Text data	`"Hello"`
Number	Integer or decimal number	`100` , `3.14`
Boolean	`true` or `false`	`true`
Null	Intentional empty value	`null`
Undefined	Variable without a value	`undefined`
BigInt	Very large integer	`123456789n`
Symbol	Unique identifier	`Symbol("id")`
Object	Collection of key-value pairs	`{ name: "Rokon" }`

JavaScript Hoisting

Hoisting is JavaScript's behavior of moving **declarations** to the top of their scope before code execution. Only declarations are hoisted, **not initializations**.

```
console.log(name);
var name = "Rokon";
```

Output:

```
undefined
```

Because JavaScript treats it like:

```
var name;
console.log(name);
name = "Rokon";
```

var Hoisting

`var` is hoisted and initialized with `undefined`.

```
console.log(message);

var message = "Hello";
```

Output:

```
undefined
```

let and const Hoisting

`let` and `const` are also hoisted, but they are **not initialized** immediately. Accessing them before declaration causes an error.

```
console.log(age);

let age = 23;
```

Output:

```
ReferenceError
```

Temporal Dead Zone (TDZ)

The **Temporal Dead Zone (TDZ)** is the time between entering a scope and declaring a `let` or `const` variable. During this period, the variable exists but cannot be accessed.

```
{
  console.log(name);

  let name = "Rokon";
}
```

Output:

```
ReferenceError
```

Key Points

- Hoisting moves declarations to the top of their scope.
- `var` is hoisted and initialized with `undefined`.
- `let` and `const` are hoisted but remain in the **Temporal Dead Zone (TDZ)** until declared.
- Accessing `let` or `const` before declaration throws a `ReferenceError`.

JavaScript ES6 Features

Type Conversion

Type Conversion means changing one data type into another.

String to Number

```
let age = "23";

console.log(Number(age)); // 23
console.log(parseInt(age)); // 23
console.log(parseFloat("23.5")); // 23.5
```

Number to String

```
let num = 100;

console.log(String(num)); // "100"
console.log(num.toString()); // "100"
```

Boolean Conversion

```
console.log(Boolean(1)); // true
console.log(Boolean(0)); // false
console.log(Boolean("Hello")); // true
console.log(Boolean("")); // false
```

Template Literals

Template Literals use **backticks ( )** instead of quotes. They allow string interpolation and multi-line strings.

Without Template Literal

```
let name = "Rokon";
let age = 23;

console.log("My name is " + name + " and I am " + age + " years old.");
```

With Template Literal

```
let name = "Rokon";
let age = 23;

console.log(`My name is ${name} and I am ${age} years old.`);
```

Multi-line String

```
let message = `Hello
Welcome to JavaScript
Have a nice day`;

console.log(message);
```

Destructuring

Destructuring extracts values from arrays or objects into variables.

Array Destructuring

```
const colors = ["Red", "Green", "Blue"];

const [first, second, third] = colors;

console.log(first); // Red
console.log(second); // Green
console.log(third); // Blue
```

Object Destructuring

```
const user = {
  name: "Rokon",
  age: 23,
  country: "Bangladesh",
};

const { name, age, country } = user;

console.log(name);
console.log(age);
console.log(country);
```

Spread Operator (...)

The **Spread Operator (** `...` **)** expands an array or object into individual elements.

Copy an Array

```
const numbers = [1, 2, 3];

const copy = [ ... numbers];

console.log(copy);
```

Merge Arrays

```
const arr1 = [1, 2];
const arr2 = [3, 4];

const result = [ ... arr1,  ... arr2];

console.log(result);
```

Copy an Object

```
const user = {
  name: "Rokon",
  age: 23,
};

const newUser = {
   ...user,
};

console.log(newUser);
```

Rest Operator (...)

The **Rest Operator (`...`)** collects multiple values into a single array.

Function Parameters

```
function sum(...numbers) {
  console.log(numbers);
}

sum(10, 20, 30, 40);
```

Output:

```
[10, 20, 30, 40]
```

Destructuring with Rest

```
const numbers = [10, 20, 30, 40, 50];

const [first, second, ...others] = numbers;

console.log(first); // 10
console.log(second); // 20
console.log(others); // [30, 40, 50]
```

Spread vs Rest

Feature	Spread Operator	Rest Operator
Purpose	Expands values	Collects values
Used In	Arrays, Objects, Function Calls	Function Parameters, Destructuring
Symbol	`...`	`...`
Example	`[...arr1, ...arr2]`	`function sum(...nums)`

Key Points

- **Type Conversion** changes one data type into another.
- **Template Literals** use backticks and `${}` for string interpolation.
- **Destructuring** extracts values from arrays and objects.
- **Spread Operator (`...`)** expands arrays or objects.
- **Rest Operator (`...`)** collects multiple values into a single array.
- Both Spread and Rest use the same syntax (`...`), but their behavior depends on where they are used.